

No-Compromise Uncertainty Quantification in a Stable Machine Learning Emulator of Subgrid Climate Physics

Katherine Frields

¹University of California at Irvine

Key Points:

- Using a confidence loss allows us to generate well-calibrated estimates of uncertainty at inference time.
- Incorporating a confidence loss does not come with sacrifices to training dynamics, offline skill, or online accuracy and stability.
- The confidence loss allows us to assess difficult-to-emulate convective regimes without the use of an expensive reference simulation.

Corresponding author: Katherine Frields, frieldskatherine@gmail.com

Abstract

The formation of clouds in a changing climate, particularly cumulus and stratocumulus clouds, remains the greatest source of uncertainty in long-term climate projections. The difficulty in modeling these structures stems from the prohibitively high computational costs associated with running models at sufficiently high resolution and appropriate complexity. These costs could potentially be circumvented by emulating high resolution subgrid physics with machine learning models that can provide the same fidelity at a fraction of the cost. However, this problem comes with its own challenges, as these emulators can make small but compounding mistakes when coupled to the host climate model over long time-scales. Fortunately, we have seen remarkable innovations in recent work, such as the integration of a microphysics constraint and the use of a modern U-net architecture. In this paper, we demonstrate well-calibrated uncertainty quantification by training the model to predict its own loss through a confidence loss. Notably, this capability comes at no apparent cost to offline or online model performance. The insights derived from analyzing this uncertainty can provide the ability to assess convective regimes that are particularly difficult to emulate online without necessitating an expensive reference simulation.

Plain Language Summary

The difficulties in modeling cloud formation has contributed significantly to the uncertainty in long-term climate projections. These challenges are rooted in the high compute power necessary to run climate models at a high enough resolution to capture the physics associated with the formation of these clouds. To circumvent this issue, researchers have attempted to replace conventional cloud resolving models with machine learning models that can run at a fraction of the compute power. However, the development of stable and high fidelity models has proven difficult, as small mistakes in the model can compound over the course of long-term climate simulations leading to significant deviations from their conventional cloud resolving model counterparts. Some recent innovations that have improved machine learning model performance include the introduction of physics-based constraints on the model, as well as a model architecture known as a U-net Convolutional Neural Network. In this paper, we introduce uncertainty quantification to our model by training it to predict the uncertainty associated with its outputs. This has provided insight into areas of uncertainty in our model, without resulting in any apparent loss in model performance.

1 Introduction

1.1 Literature Review

Clouds currently are the largest contributors to uncertainty in climate model projections (on Climate Change (IPCC), 2023). This uncertainty is largely derived from the coarse resolution of most current climate models (Schneider et al., 2017). Many modern climate models utilize parameterizations in an attempt to model subgrid atmospheric dynamics (Norman et al., 2022). However, the coarse horizontal and vertical resolution limits the ability of parameterized climate models to accurately capture turbulent updrafts that drive the cooling of small amounts of water vapor that contribute significantly to the formation of low clouds, including cumulus and stratocumulus clouds (Schneider et al., 2017). These low clouds are highly reflective, and thus, act as an important control on planet temperatures (Schneider et al., 2017). However, increasing the modern climate model resolution to a sufficient grain that would allow the modeling of cumulus cloud-level spatial scales would require, by one estimate, around 22,000 times more compute power (Norman et al., 2022). One approach has been the innovation of global cloud resolving models (GCRM) which, to an extent, are able to simulate the type of deep convection necessary for modeling these clouds (Sato et al., 2019; Taylor et al.,

2023). However, these GCRM’s have been unable to break the 1km threshold that is believed to be necessary to accurately model small-scale turbulent updrafts and are currently too computationally expensive for long-term climate modeling (Taylor et al., 2023).

One method of reducing computational cost is to emulate expensive cloud resolving models (CRM) with cheaper machine learning models. This approach is enabled by the Multiscale Modeling Frameworks (MMF), whereby separate kilometer-level CRMs can be embedded into a dynamical core. This enables the simulation of large-scale dynamics while allowing for locality in evaluation, enabling us to treat each CRM as an independent model. Critically, this allows machine learning models to train on only the inputs and outputs of a single time step of the model (Hu et al., 2024).

One of the primary challenges of developing such machine learning models is the training gap between offline and online error (Lin et al., 2023). Offline error is defined as the error of the machine learning model on the inputs and outputs of a single time step. Online error is defined as the error in the overall climate model when these machine learning models are embedded and run in a dynamic feedback loop to obtain long-term climate projections (Hu et al., 2024). Research has demonstrated that loss or gains in offline error are not necessarily reflected the online error, leading to serious difficulties optimizing these models (Lin et al., 2023; Wang et al., 2022). Furthermore, small accumulated online errors can lead to significant deviations in long-run simulations as ml-models struggle to account for coupled dynamics of the climate model and embedded ml-model (Lin et al., 2023)..

Recent research has found online success in CRM ml-emulation through the use of the U-net architecture, along with the incorporation of large-scale dynamic forcing, and the integrations of physical-constraints (Hu et al., 2024). My research builds upon this model by experimenting with various optimizers as well as the integration of uncertainty quantification using a confidence head.

1.2 Confidence Integration

A confidence head is a set of layers at the end of our model that are designed to predict our models uncertainty. This works two-fold to both increase our models sensitivity to its own uncertainty, which potentially could improve our predictions, while also providing us information on the areas in which the model has the highest uncertainty.

Our implementation of confidence loss was inspired by the AlphaFold model, a deep-learning model developed to predict protein structures (Jumper et al., 2021). The AlphaFold model utilized a confidence head to predict each output’s residual. The confidence head takes as input the last layer to the neural network, and as a target the true residuals between the predicted and target data . The difference between the predicted residuals and true residuals are then incorporated into the confidence loss function . This confidence loss is combined with prediction, to produce total loss. However, the confidence loss itself, has a very small weight, and therefore doesn’t impede the prediction of the actual values . This implementation was found to produce a reliable prediction of the outputs residuals (Hu et al., 2024).

2 Methodology

2.1 ClimSim Online

For training and evaluating our model we used the ClimSim Online Dataset and Workflow. This workflow utilizes Energy Exascale Earth System Model-Multiscale Modeling Framework version 2.1.0 (E3SM-MMF), allowing us to embed machine learning emulators in place of conventional CRMs in our climate model (Hu et al., 2024; Yu et al., 2023).

The ClimSim dataset provides the atmospheric state before and after each CRM run in the the E3SM-MMF over the course of 10 years, at 20 minute intervals. We chose to utilize the low-resolution dataset which consists of 384 columns that partition the surface of the planet in an unstructured "cubed-sphere", with each column possessing 60 vertical levels. The inputs consist of the state variables (Table 1) and the outputs consist of the subgrid tendencies (Table 5.1) (Yu et al., 2023). These inputs and outputs are then used as the input x and targets y when training and evaluating our ML emulator. The first 8 years are used for training and validation in a 7:1 split, and the last 2 years are reserved for testing (Yu et al., 2023). A scoring set is then extracted from the validation data with a 6 step stride.

The model development process can be split into the offline and the online phase. In the offline phase, the model was trained to emulate the CRM, mapping the inputs to targets at each 20 minute time step. For this research we trained four U-net models. Three of these models were implemented without the confidence head, and differed only the inclusion of either the adam, Radam, or adamW optimizer. The fourth model used the adamW optimizer as well as a confidence head. These models were then evaluated using the scoring data, comparing the the tendencies predicted by the emulator and target tendencies predicted by the CRM.

During the online phase, the model was embedded into the E3SM-MMF and evaluated on its performance against an E3SM-MMF model with conventional CRM embedding. The first step was wrapping the models so that the inputs and outputs matched those of the conventional CRM. To do this, we utilized the same pre-processing and post-processing functions we used during training and converted our outputted states to tendencies. We then ran a ClimSim online script to produce a FORTRAN script that ran the E3SM-MMF with our ml-embedding. The online performance of these models were then evaluated against that the E3SM-MMF with the conventional CRM embedding.

2.2 Model Architecture

The U-net Convolutional Neural Network architecture aims to discover both short-range and long-range patterns through the use of a constricting encoder, followed by an expanding decoder, with skip layers preserving information between the two (Hu et al. 2024). The U-net consists primarily of No Attention ResBlocks, which are composed of a normalization, activation, 1D convolutional and final normalization layer. The specific U-net architecture implemented by Hu et al. (2024) was originally adapted from Song et. al (2020), and replaced the 2D convolutional layers with 1D convolutional layers.

We modified the architecture by adding a confidence head consisting of an additional 1D convolutional layer and group normalization layer (See Figure 1). The confidence head takes as input the last layer of the decoder, and as a target the prediction loss. The model uses a huber loss function for both the prediction loss and confidence loss. These loss values are then added to get the total loss.

$$\begin{aligned} L_{prediction} &= \text{huber}(\hat{y}, y) \\ L_{confidence} &= \text{huber}(\hat{c}, L_{prediction}) \\ L_{total} &= L_{prediction} + L_{confidence} \end{aligned}$$

We will refer to our U-net model with a confidence head as a confidence U-net model and our model without a confidence head as a conventional U-net model

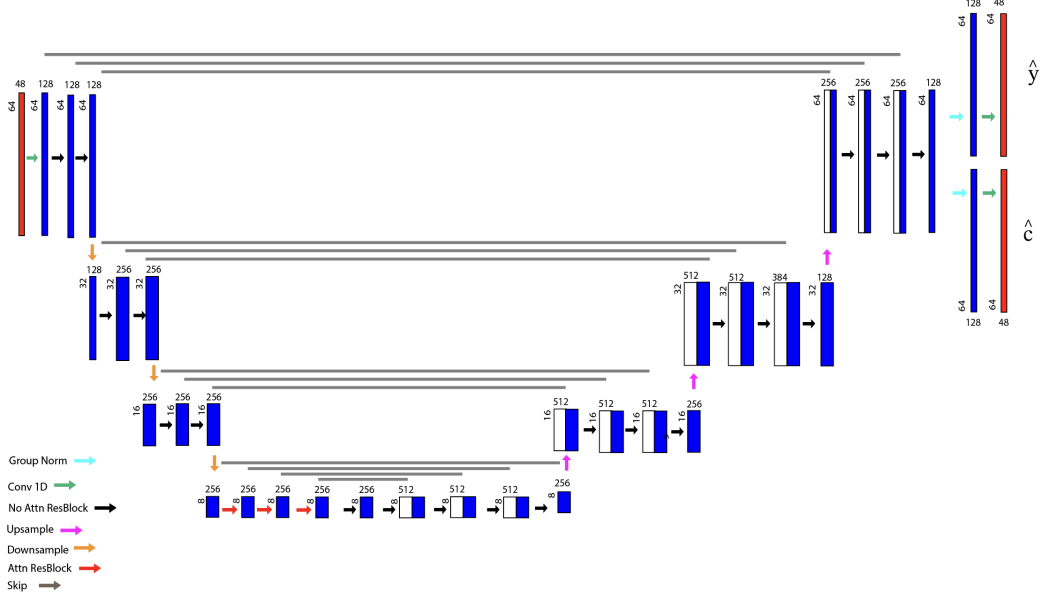


Figure 1. Confidence U-net Architecture. Each block depicts the tensor output of the previous arrow’s operation. Outputs the predicted tendencies x and the predicted loss c . Gray lines indicate the path of skip connections from the encoder to the decoder blocks.

2.3 Pre-Processing and Post-Processing

As per Hu et al. (2024), pre-processing functions were applied to the inputs and post-processing functions were applied to the outputs of emulator model for both offline training and online inference. These functions aim to improving model performance by using physical constraints, large-scale forcing, and normalizations.

One such physical constraint the amount of liquid and ice in a cloud. Since the ratio between these two amounts is a function of temperature, we can train our model to predict the change in total combined ice/liquid and then in post processing convert this to separate total liquid and total ice quantities. This ensures that the quantity of liquid and ice follows the physical constraint of temperature. Additionally, in pre-processing we removed our combined ice/liquid quantities water above the tropopause, which is defined as the vertical level at which pressure is less than 400 hPa and potential temperature is greater than 10 K/km. This has been shown to improve online stability as it prevents deep convection from forming clouds above the tropopause as, in the actual atmosphere, clouds above this layer are generally removed by sedimentation (Hu et al., 2024). Likewise, in post-processing, we likewise zeroed our moistening tendency, combined liquid-ice mixing ratio change over, zonal wind acceleration, and meridional wind acceleration for this same reason. Additionally, we transformed our specific humidity to relative humidity and restricted its values to less than 1.2, forcing our model to maintain a reasonable range of values.

Another innovation has been the inclusion of large-scale forcing between time steps during online training, allowing individual CRMs to dynamically interface with the wider model (Hu et al., 2024).

The normalizations applied to our inputs are summarized in Table 1.

2.4 Computing Resources

Our models were designed using Pytorch and leverage the NVIDIA modulus framework. All four models were trained utilizing NERSC's parallel GPU computing resources. Additionally, their outputs were configured to wandb, recording the training loss curves, and all four models converged within 24 hours.

3 Results

3.1 Comparing Optimizers

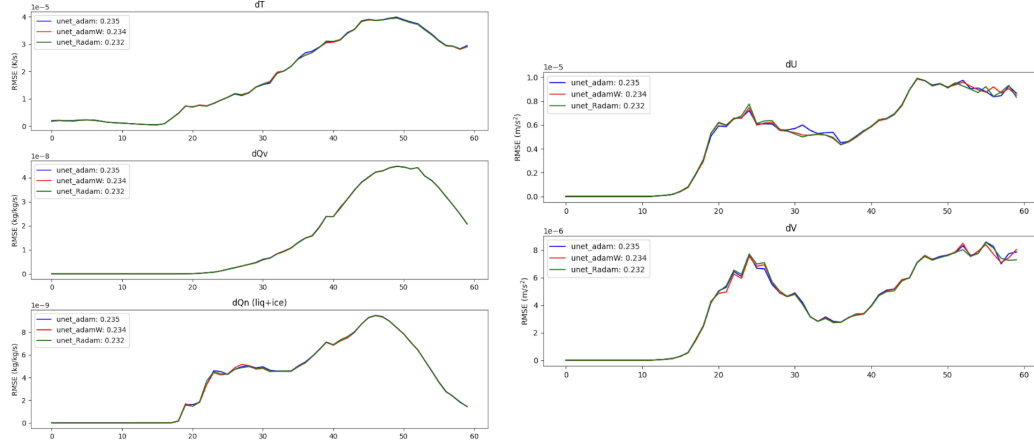


Figure 2. Offline RMSE of U-net Architecture with the adam, adamW, and Radam optimizers. Variables correspond to tendencies in 5.1, temperature tendency (dT/dt), moistening tendency ($dQ001/dt$), combined liquid ice mixing ratio (dQn/dt), zonal wind acceleration (du/dt), and meridional wind acceleration (dv/dt)

The U-net models with the adam, Radam, and adamW optimizer performed nearly identically across several offline evaluation metrics, namely RMSE, R^2 , and Bias (Figure 2, Supplementary Figures 7, 8, 9). These figures were calculated by averaging the respective evaluation statistic at each pressure level using the scoring data. These figures indicate that our models behave very similarly when utilizing the different optimizers. For the remainder of our research, we opted to utilize the adamW optimizer.

3.2 Conventional vs Confidence Model Performance

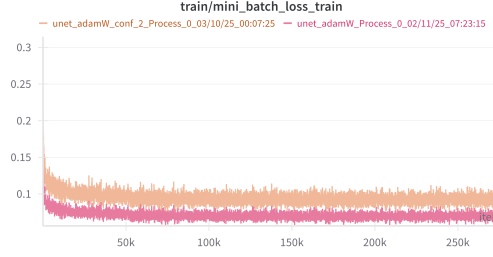


Figure 3. Training Loss Curves. Pink corresponds to conventional U-net model. Orange corresponds to confidence U-net model.

As shown in Figure 3, during training, our confidence U-net model converged to a lower loss than the our conventional U-net model. This can be explained by understanding confidence loss as a form of regularization. In training, it is expected to that confidence loss would prevent overfitting by increasing our models awareness of uncertainty. This would in turn increase the training loss, but improve model performance during scoring. With our confidence U-net model, we would therefore expect a higher training loss and possibly poorer performance on our validation data, but a better performance on our scoring data.

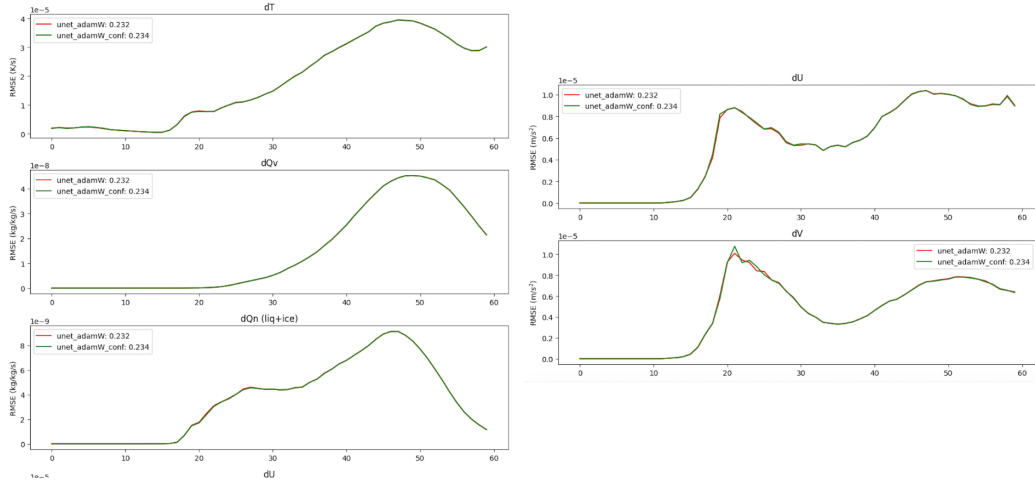


Figure 4. Offline RMSE. Red corresponds to conventional U-net model. Green corresponds to confidence U-net model. Each graph correspond to a different outputted tendency: temperature tendency (dT/dt), moistening tendency (dQ_{001}/dt), combined liquid ice mixing ratio (dQ_n/dt), zonal wind acceleration (du/dt), and meridional wind acceleration (dv/dt)

The confidence U-net model and conventional U-net model performed nearly identically across our offline evaluation metrics of RMSE, R^2 , and Bias (Figure 4, Supplementary Figures 10, 11, 12).

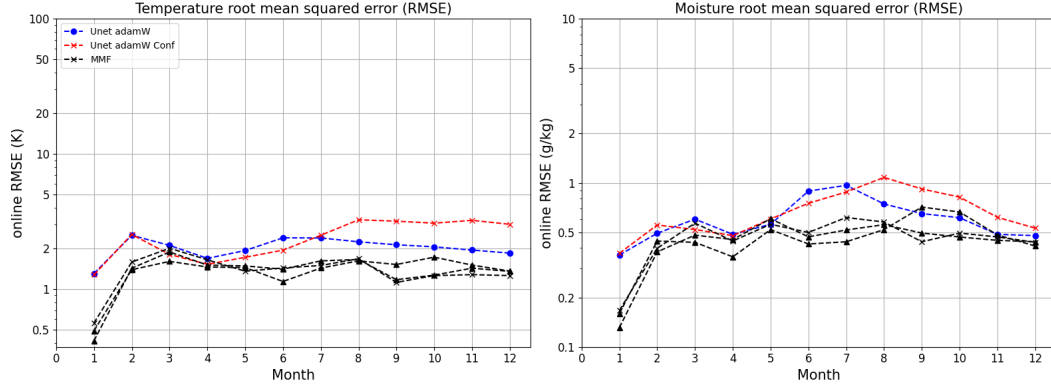


Figure 5. Online RMSE of U-net Architecture for temperature and moisture. Black corresponds to E3SM-MMF with conventional CRM embedding. Blue corresponds E3SM-MMF with conventional U-net model embedding. Red corresponds E3SM-MMF with confidence U-net model embedding

Our online performance was likewise similar across both models as measured by RMSE (Figure 5) and bias (Supplementary Figures 17, 16, 19, 18, 21, 20). This suggests that the integration of confidence loss has only minimally affected overall model performance. However, to fully validate this, further runs should be undertaken to more robustly compare the two models.

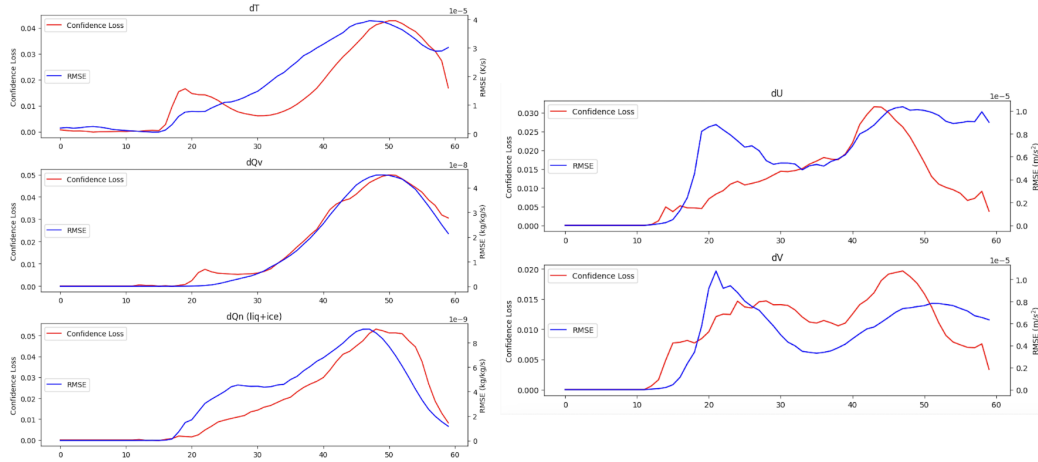


Figure 6. Offline RMSE vs predicted loss for the confidence U-net model. Red corresponds to the loss predicted by confidence head. Blue corresponds to offline RMSE.

In addition to demonstrating comparable performance, our confidence U-net model showed a strong trend between offline RMSE and confidence loss. Figure 6 depicts the predicted confidence loss vs the RMSE of the outputs for each sample of the scoring data averaged across pressure levels. Overall, though scaled differently, the trends are remarkably similar, providing evidence that the confidence loss is indeed an indicator of model uncertainty, as it is expected that areas of high uncertainty would fall in line with areas of high error. The deviations that do exist in the error and uncertainty trends are

worth exploring in further research, as they could reveal which areas of high uncertainty are due to inherent atmospheric variability, and which areas are due to poor model fitting.

4 Conclusion

Confidence loss proves to have strong potential for improving the understanding of our U-net model. The comparable online and offline performance between the confidence U-net model and conventional U-net model indicates that we can integrate a confidence head into our model without significantly sacrificing performance. Likewise, the similar trend of the offline RMSE and the predicted loss support the claim that our confidence head is accurately measuring our model’s uncertainty. Moreover, the discrepancies that do exist between model error and uncertainty open the door to further research.

Specifically, research into the specific spatial distribution of uncertainty may allow us to evaluate if our model’s uncertainty originates from poor model performance, or the natural stochasticity of our atmosphere. Another area of potential research is the use of confidence scores as a metric to inform us when to switch out for another more confident model in an ensemble. Such approach was implemented with success in the AlphaFold model and could prove useful in reducing model error during online inference (Jumper et al., 2021).

Potential improvements upon this integration of model confidence include predicting model residuals and fine-tuning the confidence loss function. By altering our confidence head to predict residuals rather than our loss, we can improve the interpretability of our uncertainty (Jumper et al., 2021). Additionally, further tuning should be performed to evaluate how much we can decrease the weighting of our confidence loss by, in order to both preserve the accuracy our models output predictions and uncertainty predictions.

On the whole, the integration of a uncertainty quantification into our U-net model provides a promising path to deepening our understanding of our model and improving model performance.

5 Supplementary Material

5.1 Input and Outputs

U-net Inputs			
Variable	Units	Description	Normalization
T	K	air temperature	(x-mean)/(max-min)
RH		relative humidity	
qn	kg/kg	combined liquid-ice mixing ratio	$1-\exp(-\lambda x)$
liq partition		liquid ice ratio	$x/(\max-\min)$
u	m/s	Zonal Wind Speed	(x-mean)/(max-min)
v	m/s	Meridonal Wind Speed	(x-mean)/(max-min)
O3	mol/mol	ozone volume mixing ratio	(x-mean)/(max-min)
CH4	mol/mol	methan volume mixing ratio	(x-mean)/(max-min)
N2O	mol/mol	nitrous oxide volume mixing ratio	(x-mean)/(max-min)
PS	Pa	surface pressure	(x-mean)/(max-min)
SOLIN	W/m^2	solar insolation	$x/(\max-\min)$
LHFLX	W/m^2	surface latent heat flux	$x/(\max-\min)$
SHFLX	W/m^2	surface sensible heat flux	$x/(\max-\min)$
TAUX	W/m^2	zonal surface stress	(x-mean)/(max-min)
TAUY	W/m^2	meridonal surface stress	(x-mean)/(max-min)
COSZRS		cosine of solar zenith angle	(x-mean)/(max-min)
ALDIF		albedo for diffuse longwave radiation	(x-mean)/(max-min)
ALDIR		albedo for direct longwave radiation	(x-mean)/(max-min)
ASDIF		albedo for diffuse shortwave radiation	(x-mean)/(max-min)
ASDIR		albedo for direct shortwave radiation	(x-mean)/(max-min)
LWUP	W/m^2	upward longwave flux	(x-mean)/(max-min)
ICEFRAC		sea-ice areal fraction	
LANDFRAC		land areal fraction	
OCNFRAC		ocean areal fraction	
SNOWHICE	m	snow depth over ice	$x/(\max-\min)$
SNOWHLAND	m	snow depth over land	$x/(\max-\min)$

Table 1. U-net Inputs

U-net Outputs

Variable	Units	Description
ptendT	K/s	temperature tendency
ptendq001	kg/kg/ks	moistening tendency
ptendqn	kg/ks/s	combined liquid-ice mixing ratio change over time
ptendu	m/s^2	zonal wind acceleration
ptendv	m/s^2	meridional wind acceleration
cam out NETSW	W/m^2	net shortwave flux at surface
cam out FLWDS	W/m^2	downward longwave flux at surface
cam out PRECSC	m/s	snow rate (liquid water equivalent)
cam out PRECC	m/s	rain rate
cam out SOLS	W/m^2	downward visible direct solar flux to surface
cam out SOLL	W/m^2	downward near-infrared direct solar flux to surface
cam out SOLSD	W/m^2	downward diffuse solar flux to surface
cam out SOLLD	W/m^2	downward diffuse near-infrared solar flux to surface

5.2 Comparing Optimizer Performance

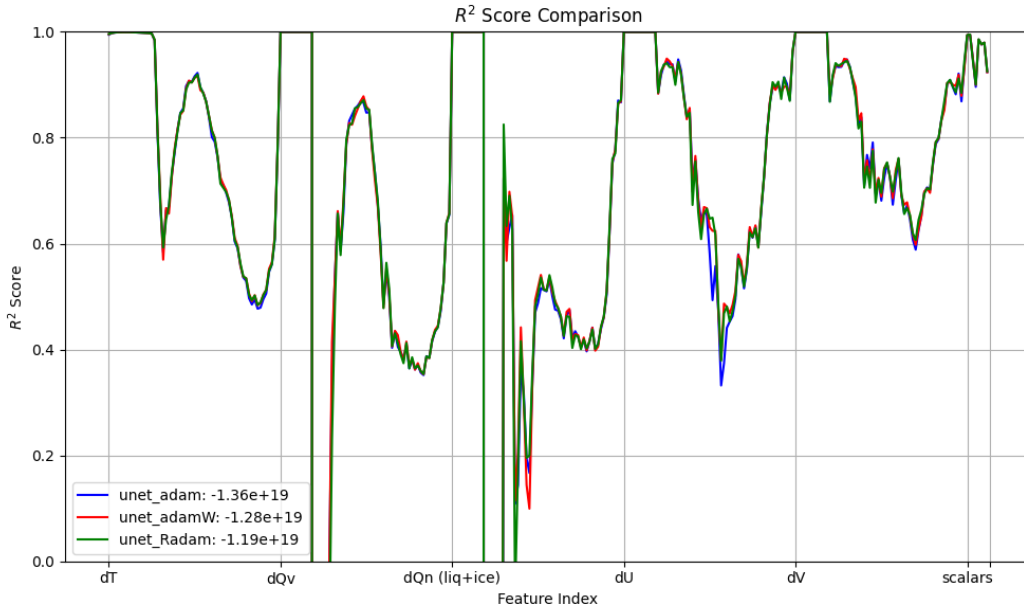


Figure 7. Offline R^2 for Various Optimizer Models

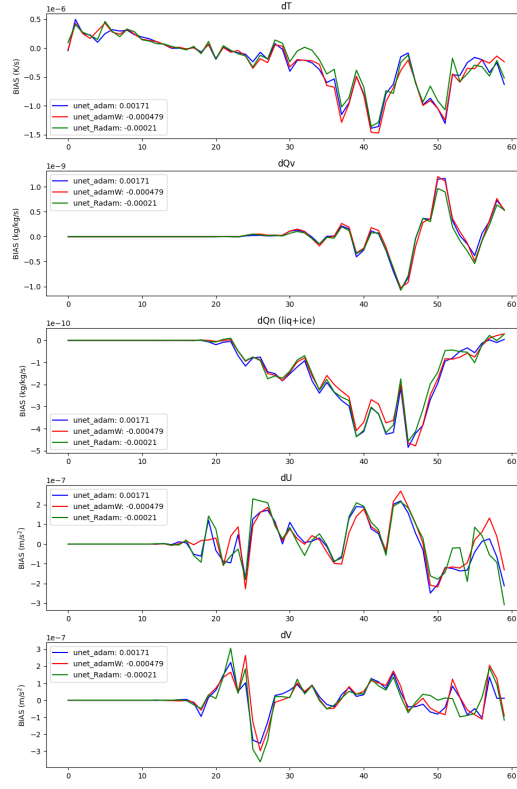


Figure 8. Offline Bias for Various Optimizer Models

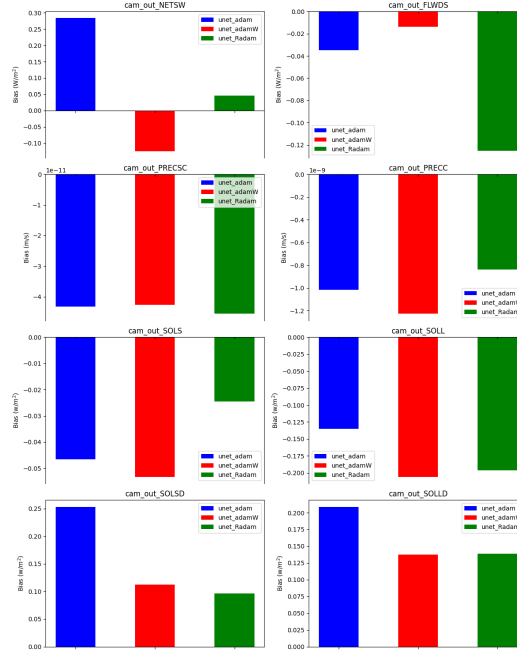


Figure 9. Offline Scalar bias for Various Optimizers

5.3 Comparing Traditional and Confidence Loss Offline Performance

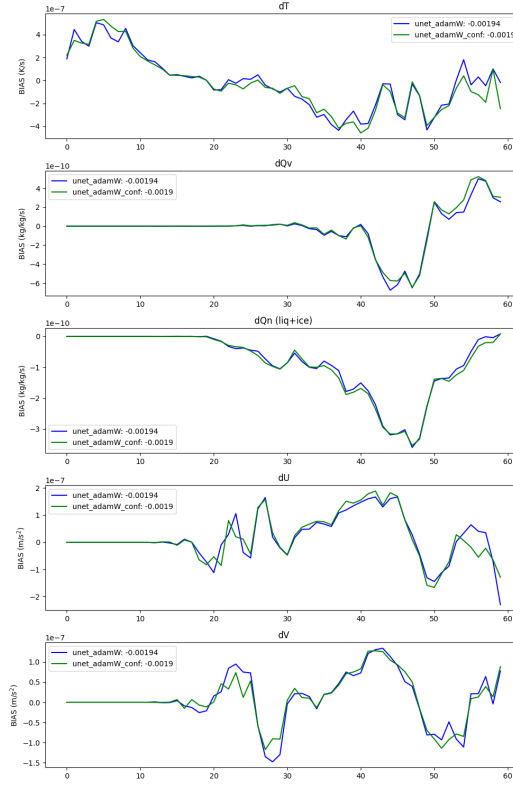


Figure 10. Offline Bias for conventional U-net model (Blue) and confidence U-net model (Green)

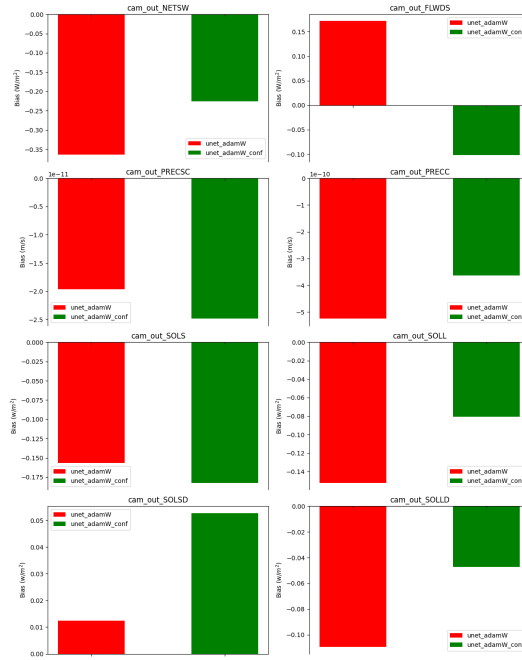


Figure 11. Offline Scalar Bias for conventional U-net model (Red) and confidence U-net model (Green).

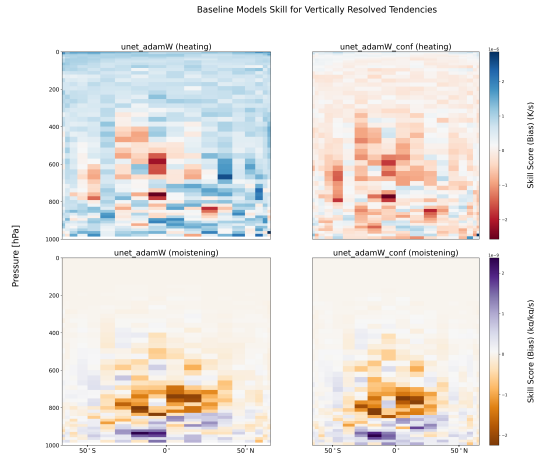


Figure 12. Offline Bias for conventional U-net model (Left) and confidence U-net model (Right) across spatial scales

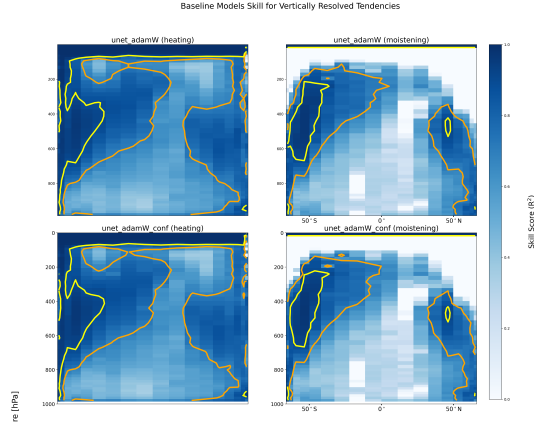


Figure 13. Offline R^2 for conventional U-net model (Top) and confidence U-net model (Bottom) across spatial scales

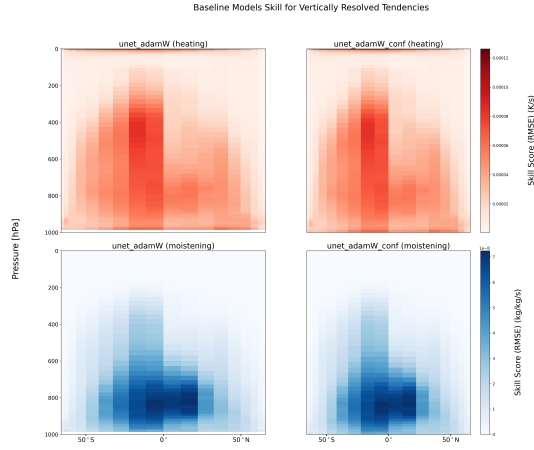


Figure 14. Offline RMSE for conventional U-net model (Left) and confidence U-net model (Right) across spatial scales

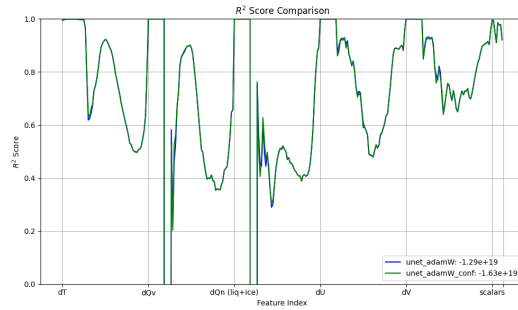


Figure 15. Offline R^2 for conventional U-net model (Blue) and confidence U-net model (Green)

5.4 Comparing Traditional and Confidence Loss Online Performance

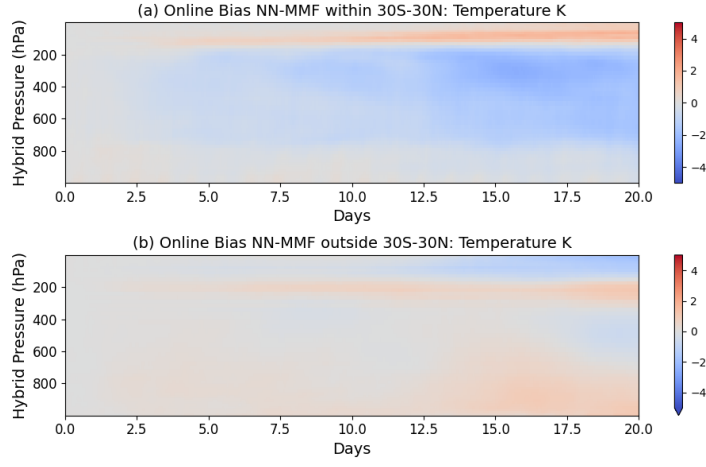


Figure 16. Online temperature bias for conventional U-net model

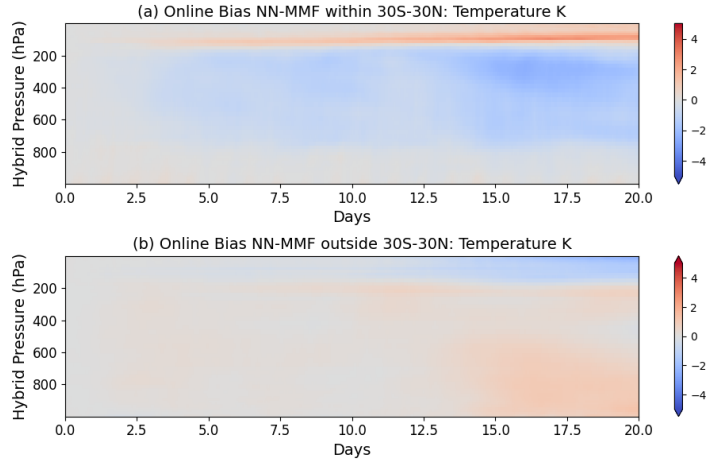


Figure 17. Online temperature bias for confidence U-net model

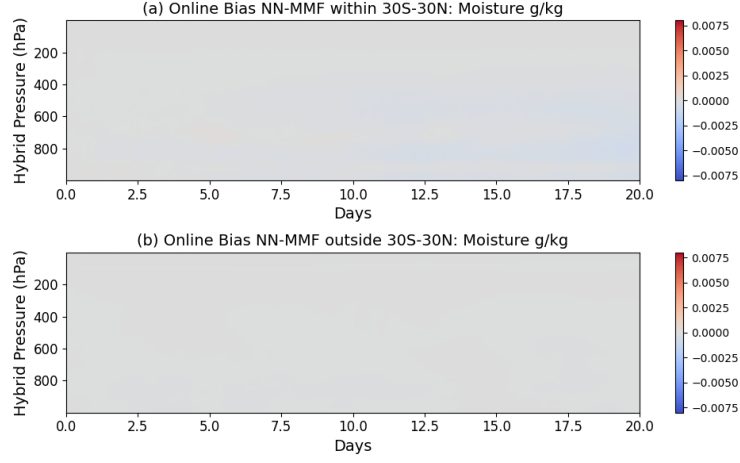


Figure 18. Online moisture bias for conventional U-net model

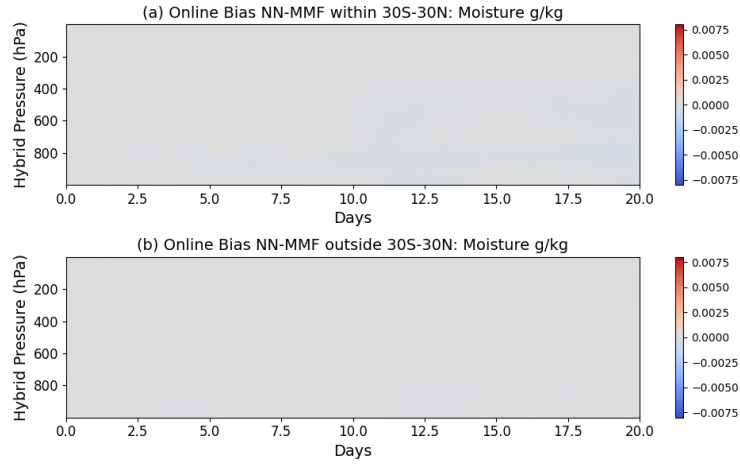


Figure 19. Online moisture bias for confidence U-net model

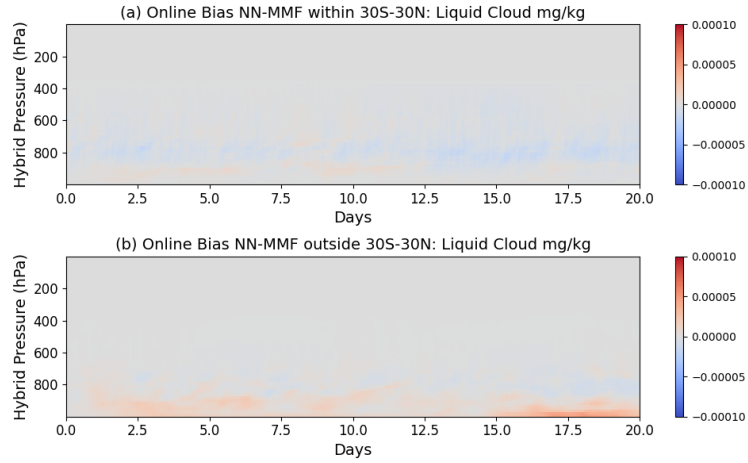


Figure 20. Online cloud-liquid bias for conventional U-net model

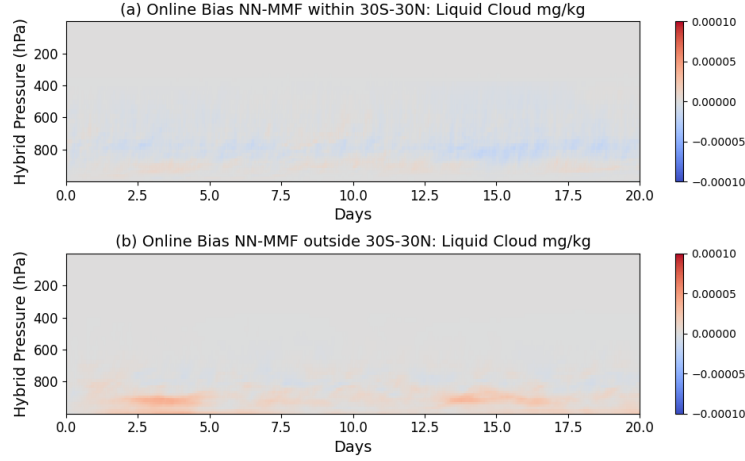


Figure 21. Online cloud-liquid bias for confidence U-net model

References

- Hu, Z., Subramaniam, A., Kuang, Z., Lin, J., Yu, S., Hannah, W. M., Brenowitz, N. D., Romero, J., & Pritchard, M. S. (2024). Stable machine-learning parameterization of subgrid processes in a comprehensive atmospheric model learned from embedded convection-permitting simulations. *arXiv [physics.ao-ph]*.
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., Bridgland, O., Meyer, C., Kohl, S. A. A., Ballard, A. J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., ... Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596, 583–589.
- Lin, J., Yu, S., Peng, L., Beucler, T., Wong-Toi, E., Hu, Z., Gentine, P., Geleta, M., & Pritchard, M. (2023). Navigating the noise: Bringing clarity to ML parameterization design with O(100) ensembles. *arXiv [physics.ao-ph]*.
- Norman, M. R., Bader, D. A., Eldred, C., Hannah, W. M., Hillman, B. R., Jones, C. R., Lee, J. M., Leung, L. R., Lyngaas, I., Pressel, K. G., Sreepathi, S., Taylor, M. A., & Yuan, X. (2022). Unprecedented cloud resolution in a GPU-enabled full-physics atmospheric climate simulation on OLCF’s summit supercomputer. *Int. J. High Perform. Comput. Appl.*, 36, 93–105.
- on Climate Change (IPCC), I. P. (2023). Technical summary. In *Climate change 2021 – the physical science basis* (pp. 35–144). Cambridge University Press.
- Satoh, M., Stevens, B., Judt, F., Khairoutdinov, M., Lin, S. J., Putman, W. M., & Düben, P. (2019). Global cloud-resolving models.
- Schneider, T., Teixeira, J., Bretherton, C. S., Brient, F., Pressel, K. G., Schär, C., & Siebesma, A. P. (2017). Climate goals and computing the future of clouds. *Nature Climate Change*, 7, 3–5.
- Taylor, M. A., Caldwell, P. M., Bertagna, L., Clevenger, C., Donahue, A. S., Foucar, J. G., Guba, O., Hillman, B. R., Keen, N., Krishna, J., Norman, M. R., Sreepathi, S., Terai, C. R., White, J. B., Wu, D., Salinger, A. G., McCoy, R. B., Leung, L. R., & Bader, D. C. (2023). The simple cloud-resolving E3SM atmosphere model running on the frontier exascale system. *International Conference for High Performance Computing, Networking, Storage and Analysis, SC*.
- Wang, X., Han, Y., Xue, W., Yang, G., & Zhang, G. J. (2022). Stable climate simulations using a realistic general circulation model with neural network parameterizations for atmospheric moist physics and radiation processes. *Geosci. Model Dev.*, 15, 3923–3940.

Yu, S., Hu, Z., Subramaniam, A., Hannah, W., Peng, L., Lin, J., Bhourri, M. A., Gupta, R., Lütjens, B., Will, J. C., Behrens, G., Busecke, J. J. M., Loose, N., Stern, C. I., Beucler, T., Harrop, B., Heuer, H., Hillman, B. R., Jenney, A., . . . Pritchard, M. (2023). ClimSim-online: A large multi-scale dataset and framework for hybrid ML-physics climate emulation.